

Precision-aware fixed-point emulation of Grover's algorithm — 해설·정리본

항목	내용
원문 제목	Precision-aware fixed-point emulation of Grover's algorithm: asymptotic error bounds and design g
저자	Seonghyun Choi, Kyeongwon Lee, Jongin Choi, Chaebin Jung, Woojoo Lee (교신) — 중앙대학교 지능
게재지	Quantum Information Processing (2026) 25:214 — Springer, Open Access (CC BY-NC-ND 4.0)
DOI	https://doi.org/10.1007/s11128-026-05235-9
투고/게재	2025-06-16 접수 / 2026-05-26 게재확정 / 2026-06-12 온라인 공개
분량	원문 28쪽 (본문 24쪽 + 참고문헌 4쪽), 그림 7개 · 표 3개 · 알고리즘 2개 · 수식 63개
원문 파일	../papers/2. Grover 논문 (LPSoC)/Precision-aware fixed-point emulation of Grover's a
본 정리본	원문의 약 35~40% 분량 (요약·재구성이며 원문 대체본이 아닙니다)

한눈에 보기

고정소수점(fixed-point)으로 그로버 알고리즘을 에뮬레이션할 때, 소수부 비트 f 가 부족해서 생기는 **반올림 오차(truncation error)** 가 반복(iteration)을 거치며 어떻게 누적되는지를 끝까지 계산해 낸 논문입니다. 핵심 결론은 최종 측정 확률분포의 오차가 $O(2^{n-f})$ 로 스케일한다는 것 — **큐비트를 1개 늘리면 오차가 2배, 소수부 비트를 1개 늘리면 오차가 절반**입니다. 저자들은 이 관계를 상수까지 실험으로 맞춰서, 원하는 정확도 $\ell_2^{\max}$ 를 넣으면 필요한 최소 소수부 비트가 바로 나오는 닫힌 형태의 공식 $f_{\min} = \lceil n - \log_2 \ell_{2,\max} - 1.03 \rceil$ 을 제시합니다. 이 프로젝트의 Verilog IP에 그대로 대입할 수 있는, 사실상 “비트폭 설계 매뉴얼”에 해당하는 논문입니다.

목차

1. 서론 — 왜 정밀도가 설계 변수인가 (원문 p.2-4)
2. 배경 지식 (원문 p.4-11)
 - 2.1 양자 정보와 게이트 연산 (p.4-6)
 - 2.2 그로버 알고리즘 (p.6-8)
 - 2.3 고정소수점 산술과 반올림 오차 (p.8-11)
3. 반올림 오차의 이론적 분석 (원문 p.11-18)
 - 3.1 두 진폭 구조의 보존 증명 (p.11-12)
 - 3.2 오차 누적의 수식화 ★ (p.12-16)
 - 3.3 측정 확률분포에 미치는 영향 ★ (p.16-18)
4. 실험 검증과 정밀도 설계 지침 (원문 p.18-24)
 - 4.1 오차 공식의 검증 (p.18-23)
 - 4.2 목표 정확도를 위한 최적 정밀도 선택 ★★ (p.23-24)
5. 결론과 한계 (원문 p.24-25)
6. 이 프로젝트에 어떻게 쓰이나 ★★★

1. 서론 — 왜 정밀도가 설계 변수인가

원문 p.2-4

양자 하드웨어는 아직 확장성과 오류정정 문제로 대규모 fault-tolerant 단계에 도달하지 못했고, 그래서 연구자들이 알고리즘을 개발·검증할 실물 하드웨어에 접근하기가 어렵습니다. 이 공백을 메우는 것이 고전 컴퓨터 위에서 양자 회로를 흉내 내는 **QC 에뮬레이터**입니다. 에뮬레이터는 양자 상태를 벡터로, 게이트(gate)를 행렬로 모델링하는데, 바로 이 지점에서 문제가 생깁니다. 고전 하드웨어의 수치 오차 — 부동소수점(floating-point)이든 고정소수점이든 — 가 에뮬레이션된 연산의 충실도(fidelity)를 직접 갉아먹기 때문입니다.

저자들이 강조하는 논점은 이렇습니다. 상태벡터(state vector) 에뮬레이션의 메모리 비용은 **진폭(amplitude)의 개수 $\times$ 워드 길이(word length)**로 정해집니다. 진폭 개수가 2^n 으로 지수적으로 늘어나는 건 고전 에뮬레이션의 숙명이지만, **워드 길이는 설계자가 고르는 값**입니다. 그러니 정밀도는 메모리 풋프린트, 데이터패스 비용, 정확도를 한꺼번에 결정하는 핵심 구현 파라미터입니다. 그런데 기존 연구들은 “고정소수점 양자화 오차가 회로 규모에 따라 커진다”는 관찰까지는 했어도, 대개 **더 높은 정밀도의 부동소수점으로 갈아타는 것으로** 대응했을 뿐이고, 산술 정밀도를 에뮬레이터 정확도의 명시적 한계로 번역해 주는 체계적·알고리즘 특화 연구는 드물었습니다. 특히 “주어진 알고리즘과 오차 허용치에 대해 고정소수점 정밀도가 **최소 얼마나** 필요한가”에 답하는 연구가 없었다는 것이 이 논문의 출발점입니다.

여기서 이 논문의 목표를 정확히 짚어 둘 필요가 있습니다. 저자들이 하려는 것은 **그로버의 이상적인 최종 상태를 달린 형태로 재현하는 것이 아닙니다**. 그건 이미 (원문 Eq. 18)로 알려져 있습니다. 대신 **고정소수점 에뮬레이터의 반복 단위 실행을** 분석합니다 — 반복되는 확산 스케일링(diffusion-scaling) 단계마다 반올림이 발생하고, 그것이 그로버 반복을 거치며 누적되는 과정 말입니다. 이 구분이 논문 전체를 관통하며, 4장에서 왜 굳이 반복 루프를 돌려 검증하는지의 이유이기도 합니다.

대상 알고리즘으로 그로버를 고른 이유도 분명합니다. 그로버가 유명해서가 아니라, **달린 형태의 분석이 가능한 구조**이기 때문입니다. 그로버는 (1) 상태 진화가 **2차원 부분공간(two-amplitude subspace)** 안에 머물고, (2) **같은 연산자 G 가 반복 적용**됩니다. 이 두 가지 구조적 규칙성 덕분에 산술 오차 전파를 형식적으로 추적해서 구체적인 설계 규칙으로 번역할 수 있습니다. 저자들은 이 연구가 임의의 양자 알고리즘으로 일반화된다고 주장하지 않으며, “구조적이고 실용적으로 의미 있는 사례에서의 엄밀한 알고리즘 특화 정밀도 분석”이라고 스스로 범위를 한정합니다.

고정소수점을 택한 이유는 하드웨어 효율 때문만이 아니라, 상태 저장이 이미 지수적으로 늘어나는 상황에서 **진폭 하나당 워드 길이를 통제해야 하는 현실적 필요** 때문입니다. 하드웨어 지향 QC 에뮬레이터에서 고정소수점 연산기가 부동소수점 대비 로직 사용량·전력·지연을 크게 줄여 준다는 것은 이미 여러 선행 연구가 보인 바 있습니다. 그리고 중요한 전제: **이 논문은 오라클(oracle)과 확산 연산이 이상적이라고 가정하고, 반올림을 독립된 산술 오차원으로 분리**합니다. 소자 잡음이나 불완전한 양자 게이트는 모델링하지 않습니다.

논문의 기여는 세 가지입니다. 첫째 **수치 오차 분석** — 반올림 오차가 측정 확률분포에 어떻게 나타나는지를 ℓ_2 거리로 유도합니다. 둘째 **실험적 검증** — 이론 예측과 실제 에뮬레이터 출력을 비교합니다. 셋째 **설계 지침** — 목표 오차와 필요 정밀도를 잇는 달린 형태 공식을 제시합니다. 이 셋 중 설계자에게 실질적으로 중요한 것은 단연 세 번째이고, 그것이 4.2절의 (원문 Eq. 63)입니다.

2. 배경 지식

원문 p.4-11

2.1 양자 정보와 게이트 연산

원문 p.4-6

이미 이 프로젝트의 1장~4장에서 다룬 내용이라 빠르게 넘어갑니다. 단일 큐비트(qubit)의 기저 상태는 단위 열벡터 $|0\rangle = [1 \ 0]^T$, $|1\rangle = [0 \ 1]^T$ 이고(원문 Eq. 1), 중첩(superposition)은

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \|\alpha\|^2 + \|\beta\|^2 = 1$$

로 씁니다(원문 Eq. 2). n 큐비트 시스템은 2^n 개의 기저 상태를 가지며 2^n 차원 열벡터로 모델링됩니다(원문 Eq. 3):

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle = [\alpha_0 \ \alpha_1 \ \cdots \ \alpha_{2^n-1}]^T.$$

측정(measurement) 시 $|i\rangle$ 이 나올 확률은 $P(|\psi\rangle \rightarrow |i\rangle) = \|\alpha_i\|^2$ 이고(원문 Eq. 4), 정규화 조건은

$$\| |\psi\rangle \| = \sqrt{\sum_{i=0}^{2^n-1} \|\alpha_i\|^2} = 1 \quad (\text{원문 Eq. 5})$$

입니다. 이 Eq. (5)는 뒤에서 정수부 비트폭을 정할 때 다시 등장하니 기억해 두세요. 게이트는 이 정규화를 보존해야 하므로 반드시 유니터리(unitary) 행렬입니다.

원문 Table 1은 대표적인 게이트들의 행렬을 정리합니다. 새로운 내용은 없지만 참조용으로 옮깁니다.

게이트	기호	유니터리 행렬	게이트	기호	유니터리 행렬
Hadamard	H	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	Phase-shift	P	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
Pauli	X	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	T	T	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
	Y	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	Controlled-NOT	$CNOT$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
	Z	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$			

병렬로 적용되는 단일 큐비트 게이트들은 크로네커 곱(Kronecker product)으로 합쳐집니다: $U = U_1 \otimes U_2 \otimes \dots \otimes U_n$ (원문 Eq. 6).

2.2 그로버 알고리즘

원문 p.6-8

그로버는 $f(x) = 1$ 을 만족하는 x 를 찾는 절차입니다. S 를 해(solution)의 집합, $n_s = |S|$ 를 해의 개수라 하면(이 프로젝트 9장에서 쓰던 M 과 같습니다)

$$f(x) = \begin{cases} 1 & x \in S, \\ 0 & \text{otherwise,} \end{cases} \quad (\text{원문 Eq. 7})$$

이고, 알고리즘은 균등중첩을 만든 뒤 그로버 연산자 G 를 k 번 적용합니다:

$$|\psi_{\text{final}}\rangle = G^k |u\rangle, \quad G = (H^{\otimes n} Z_{\text{or}} H^{\otimes n}) (Z_f). \quad (\text{원문 Eq. 8})$$

$$k = \left\lceil \frac{\pi}{4} \sqrt{\frac{2^n}{n_s}} \right\rceil \quad (\text{원문 Eq. 9})$$

여기서 $\lceil \cdot \rceil$ 은 **가장 가까운 정수로 반올림**입니다(내림이 아닙니다 — 뒤에서 우리 코드와 비교할 때 걸리는 지점입니다). 균등중첩 초기 상태는

$$|u\rangle = H^{\otimes n} |0^n\rangle = \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} |i\rangle. \quad (\text{원문 Eq. 10})$$

G 는 두 부분으로 쪼개집니다. **오라클** Z_f 는 해에 해당하는 기저 상태의 위상을 뒤집습니다 — 즉 진폭 계수의 부호를 반전시킵니다(위상 킥백(phase kickback)의 결과물이죠):

$$Z_f |x\rangle = (-1)^{f(x)} |x\rangle = \begin{cases} -|x\rangle & x \in S, \\ |x\rangle & x \notin S. \end{cases} \quad (\text{원문 Eq. 11})$$

확산 연산자(diffusion operator) 는 모든 2^n 개 진폭을 평균에 대해 반사(평균 반사)합니다:

$$H^{\otimes n} Z_{\text{or}} H^{\otimes n} = 2|u\rangle\langle u| - I_n = \begin{bmatrix} 2^{-n+1} & \dots & 2^{-n+1} \\ \vdots & \ddots & \vdots \\ 2^{-n+1} & \dots & 2^{-n+1} \end{bmatrix} - I_n, \quad (\text{원문 Eq. 12})$$

$$Z_{\text{or}}|x\rangle = 2|0^n\rangle\langle 0^n|x\rangle - |x\rangle = \begin{cases} -|x\rangle & x \neq 0, \\ |x\rangle & \text{otherwise.} \end{cases} \quad (\text{원문 Eq. 13})$$

Eq. (12)의 행렬 형태가 이 논문 전체의 열쇠입니다. 모든 성분이 2^{-n+1} 인 행렬을 곱한다는 것은 곧 “모든 진폭을 다 더한 뒤 2^{-n+1} 을 곱하고, 원래 진폭을 빼라”는 뜻입니다. 하드웨어에서 2^{-n+1} 곱하기는 $(n-1)$ 비트 우측 시프트로 구현되고 — 바로 여기서 소수부 비트가 잘려 나갑니다. 논문이 추적하는 반올림 오차는 사실상 전부 이 한 줄에서 발생합니다.

이제 두 개의 정규직교 벡터를 정의합니다:

$$|\psi_S\rangle = \frac{1}{\sqrt{n_s}} \sum_{\substack{i=0 \\ i \in S}}^{2^n-1} |i\rangle, \quad |\psi_{NS}\rangle = \frac{1}{\sqrt{2^n - n_s}} \sum_{\substack{i=0 \\ i \notin S}}^{2^n-1} |i\rangle. \quad (\text{원문 Eq. 14, 15})$$

이 둘은 서로 직교하고 각각 단위 길이이므로 2차원 부분공간의 정규직교 기저를 이룹니다. 초기 상태를 이 부분공간에서 각도 형태로 쓰면

$$|u\rangle = \sin(\theta)|\psi_S\rangle + \cos(\theta)|\psi_{NS}\rangle, \quad \theta = \sin^{-1}\left(\sqrt{\frac{n_s}{2^n}}\right). \quad (\text{원문 Eq. 16, 17})$$

G 는 이 단위원 위에서 2θ 만큼의 회전입니다. k 번 적용하면 $2k\theta$ 회전하므로

$$|\psi_{\text{final}}\rangle = G^k|u\rangle = \sin((2k+1)\theta)|\psi_S\rangle + \cos((2k+1)\theta)|\psi_{NS}\rangle. \quad (\text{원문 Eq. 18})$$

Eq. (9)대로 k 를 고르면 $(2k+1)\theta$ 가 $\pi/2$ 에 가까워져 $|\psi_S\rangle$ 의 진폭이 최대가 됩니다. 잔여 각도 오차를 $\delta = \frac{\pi}{2} - (2k+1)\theta$ 라 두면 $|\delta| < \theta$ 임을 보일 수 있고, 따라서

$$|\cos((2k+1)\theta)| = |\sin \delta| = \sin |\delta| < \sin \theta \quad (\text{원문 Eq. 19})$$

즉 비해(nonsolution) 진폭은 초기 진폭 $\sin \theta$ 보다 반드시 작아집니다. 이 부등식은 3.3절에서 $|\alpha_{k,FP,NS}| = O(2^{-n})$ 을 유도하는 데 그대로 재사용되므로, 장식이 아니라 필요해서 넣어 둔 보조 결과입니다.

원문 Fig. 1(p.8)은 진폭 막대그래프 3장으로 G 한 번을 보여 줍니다. 왼쪽은 균등중첩(모든 막대가 같은 높이, $m = s$ 표시), 가운데는 오라클 통과 후(해 막대만 아래로 뒤집혀 음수), 오른쪽은 확산 후(평균선 $\frac{1}{2^n} \sum \psi[m]$ 을 기준으로 반사되어 해 막대만 경충 솟고 나머지는 조금씩 낮아짐)입니다. 이 프로젝트 9장의 그림과 같은 이야기입니다. 원문 Fig. 2(p.9)는 같은 과정을 $|\psi_{NS}\rangle$ - $|\psi_S\rangle$ 평면의 회전으로 그린 것으로, $|\psi_0\rangle$ 이 θ 에서 출발해 매번 2θ 씩 반시계로 돌아 $|\psi_k\rangle = |\psi_{\text{final}}\rangle$ 이 $|\psi_S\rangle$ 축(각도 $\pi/2$)에 δ 만큼 못 미친 채 멈추는 모습을 보여 줍니다.

2.3 고정소수점 산술과 반올림 오차

원문 p.8-11

고정소수점은 정수부와 소수부에 **고정된** 비트폭을 할당해 실수를 표현합니다(원문 Fig. 3, p.9 — 상위 i 비트가 2^{i-1} 부터 2^0 까지, 하위 f 비트가 2^{-1} 부터 2^{-f} 까지를 담당하는 비트 배열 그림). 정수부 폭을 i , 소수부 폭을 f 라 하면 표현 가능한 범위와 해상도(resolution)는

$$-(2^{i-1} - 2^{-f}) \leq r \leq 2^{i-1} - 2^{-f}, \quad \Delta r = 2^{-f}. \quad (\text{원문 Eq. 20, 21})$$

덧셈과 곱셈이 왜 다른가. 원문 Fig. 4(p.10)가 이걸 그림으로 설명합니다. 덧셈에서는 정수부가 할당 비트를 넘으면 오버플로가 날 수 있지만 **소수부 정밀도는 변하지 않습니다** — 즉 덧셈은 추가 반올림 오차를 만들지 않습니다. 반면 곱셈은 정수부·소수부가 모두 원래 폭을 넘어 자릅니다. 두 개의 f 비트 소수부를 곱하면 결과의 소수부는 2^{-2f} 까지 필요해지는데 가용한 건 2^{-f} 뿐이므로, **하위 비트가 잘려 나가며 반올림 오차가 발생합니다.** 원문 Fig. 5(p.10)는 우측 시프트가 하위 소수부 비트를 밀어내는 모습을 따로 그려, “잘려 나간 소수부 비트가 곧 반올림 오차의 원천”임을 못 박습니다.

정수부 비트폭은 왜 $i = n$ 인가. 그로버에서 가장 커질 수 있는 중간값은 확산 연산의 **스케일링 전 누산합**입니다. 코시-슈바르츠 부등식(Cauchy-Schwarz)과 정규화 조건 Eq. (5)를 쓰면

$$\left| \sum_{m=0}^{2^n-1} \psi[m] \right| \leq \sqrt{2^n} \|\psi\|_2 = 2^{n/2}$$

이므로, 저자들은 **보수적으로 $i = n$ 으로 잡습니다.** (논문은 이것이 최소폭이라고 주장하지 않습니다 — “고려한 파라미터 영역에서 오버플로가 없기에 충분한 선택”이라고만 합니다. 실제로 $2^{n/2}$ 를 담는 데 필요한 정수부는 대략 $n/2 + 1$ 비트이니 여유가 꽤 있습니다.) 오라클 Eq. (11)은 부호만 뒤집으므로 크기를 키우지 않습니다. 결국 **오버플로는 설계상 배제되고, 남은 오차원은 소수부 f 로 인한 반올림뿐**입니다. 이것이 3장 분석의 전제입니다.

스케일링, 즉 2의 거듭제곱 곱하기는 비싼 곱셈기(multiplier — 하드웨어 자원을 많이 먹는 그 곱셈기) 대신 **비트 시프트**로 처리할 수 있습니다. a 비트 좌측 시프트는 2^a 곱하기, 우측 시프트는 2^{-a} 곱하기입니다. 그리고 우측 시프트가 하위 소수부 비트를 제거하면서 반올림 오차를 만듭니다.

수식으로 정리하면, 실수 r 이 소수부 f 비트의 고정소수점 r_{FP} 로 표현될 때 반올림 오차 ϵ 는

$$r = r_{FP} + \epsilon, \quad \epsilon = r \bmod 2^{-f}. \quad (\text{원문 Eq. 22})$$

즉 r_{FP} 는 2^{-f} 의 배수이고, ϵ 은 그 정밀도 한계를 넘어가는 잔여분입니다. **항상 $0 \leq \epsilon < 2^{-f}$ 이며 부호가 한쪽(양수)으로 쏠린다**는 점이 중요합니다 — 이것이 truncation(버림)이지 반올림(round-to-nearest)이 아니라는 뜻이고, 오차가 서로 상쇄되지 않고 **한 방향으로 누적**되는 이유입니다.

3. 반올림 오차의 이론적 분석

원문 p.11-18

3.1 두 진폭 구조의 보존 증명

원문 p.11-12

이 절의 결론 한 줄: **그로버의 상태는 언제나 딱 두 종류의 진폭 값만 갖습니다.** 초기 상태가 모든 기저 상태에 같은 진폭을 주고, G 가 해 상태들을 대칭적으로 다루기 때문에, 어느 반복 단계에서도 모든 해 상태의 진폭은 서로 같고 모든 비해 상태의 진폭도 서로 같습니다(값은 서로 다르지만).

$$\psi[m] = \begin{cases} \alpha_S, & m \in S, \\ \alpha_{NS}, & \text{otherwise,} \end{cases} \quad m = 0, 1, \dots, 2^n - 1. \quad (\text{원문 Eq. 23})$$

이게 왜 중요하냐면, 2^n 차원 벡터를 추적하는 문제가 스칼라 2개를 추적하는 문제로 줄어들기 때문입니다. 이 축약이 없으면 오차 누적을 닫힌 형태로 풀 수 없습니다. 저자들은 이를 수학적 귀납법으로 증명합니다.

기저 단계. $|\psi_0\rangle = H^{\otimes n}|0^n\rangle$ 은 모든 진폭이 $\psi_0[m] = 1/\sqrt{2^n}$ 이므로(원문 Eq. 24) $\alpha_S = \alpha_{NS} = 1/\sqrt{2^n}$ 인 Eq. (23) 형태입니다.

귀납 단계. l 번째 반복 후 $\psi_l[m]$ 이 $\alpha_{l,S} / \alpha_{l,NS}$ 두 값만 갖는다고 가정합니다(원문 Eq. 25). 먼저 $|\psi'_l\rangle = Z_f|\psi_l\rangle$ 은 해 진폭에만 -1 을 곱하므로 여전히 두 값($-\alpha_{l,S}, \alpha_{l,NS}$)만 갖습니다. 여기에 확산을 적용하면

$$|\psi_{l+1}\rangle = G|\psi_l\rangle = (H^{\otimes n}Z_fH^{\otimes n})|\psi'_l\rangle = 2^{-n+1} \begin{bmatrix} \sum_{h=0}^{2^n-1} \psi'_l[h] \\ \vdots \\ \sum_{h=0}^{2^n-1} \psi'_l[h] \end{bmatrix} - |\psi'_l\rangle \quad (\text{원문 Eq. 27})$$

성분별로 보면

$$\psi_{l+1}[m] = 2^{-n+1} \sum_{h=0}^{2^n-1} \psi'_l[h] - \psi'_l[m] = \begin{cases} 2^{-n+1} \sum_h \psi'_l[h] + \alpha_{l,S}, & m \in S, \\ 2^{-n+1} \sum_h \psi'_l[h] - \alpha_{l,NS}, & \text{otherwise.} \end{cases} \quad (\text{원문 Eq. 28})$$

첫 항(누산합의 스케일링)은 m 에 **무관한 공통값**이고, 빼는 항만 $m \in S$ 인지에 따라 두 값 중 하나입니다. 따라서 결과도 두 값뿐입니다. 귀납법에 의해 모든 반복에서 Eq. (23)이 유지됩니다. ($m \in S$ 일 때 부호가 $+\alpha_{l,S}$ 인 것은 오라클이 이미 부호를 뒤집어 놔기 때문입니다.)

3.2 오차 누적의 수식화 ★

원문 p.12-16

이 절이 논문의 심장입니다. 표기가 뻑뻑하니 먼저 규칙부터 정리합니다.

표기	의미
' (프라임 1개)	오라클 Z_f 적용 직후 의 양
" (프라임 2개)	그로버 연산자 G 전체를 적용한 후 의 양
아래첨자 FP	고정소수점 양
아래첨자 scaled	확산 스케일링 단계의 출력
아래첨자 S / NS	해(solution) / 비해(nonsolution) 진폭
$e_l[m], e_l\rangle$	전체 상태 표기의 오차 (2^n 차원 벡터 또는 그 성분)
$\epsilon_{l,S}, \epsilon_{l,NS}$	두 진폭 표현으로 축약된 스칼라 오차 값

기본 분해. $|\psi_l\rangle$ 을 l 번 반복 후의 정확한 상태라 하고, 각 진폭을 고정소수점으로 표현한 벡터를 $|\psi_{l,FP}\rangle$, l 번째까지 **누적된** 반올림 오차를 $|e_l\rangle$ 라 하면

$$|\psi_l\rangle = |\psi_{l,FP}\rangle + |e_l\rangle. \quad (\text{원문 Eq. 29})$$

3.1절에 의해 $|\psi_{l,FP}\rangle$ 도 $|e_l\rangle$ 도 두 값만 갖습니다(원문 Eq. 30, 31):

$$\psi_{l,FP}[m] = \begin{cases} \alpha_{l,FP,S}, & m \in S, \\ \alpha_{l,FP,NS}, & \text{otherwise,} \end{cases} \quad e_l[m] = \begin{cases} \epsilon_{l,S}, & m \in S, \\ \epsilon_{l,NS}, & \text{otherwise.} \end{cases}$$

상태 갱신. G 는 선형이므로 상태와 오차에 각각 따로 작용시킬 수 있습니다:

$$|\psi_{l+1}\rangle = G|\psi_{l,FP}\rangle + G|e_l\rangle = |\psi''_{l,FP}\rangle + |e''_l\rangle. \quad (\text{원문 Eq. 32})$$

핵심 계산 — 누산합의 축약. $|\psi'_{l,FP}\rangle$ 이 두 값만 가지므로 2^n 항의 합을 n_s 로 묶어낼 수 있습니다. 이것이 이 논문에서 가장 중요한 대수 조작입니다:

$$2^{-n+1} \sum_{h=0}^{2^n-1} \psi'_{l,FP}[h] = 2^{-n+1} \left((2^n - n_s) \alpha_{l,FP,NS} - n_s \alpha_{l,FP,S} \right) = 2 \alpha_{l,FP,NS} - 2^{-n+1} n_s (\alpha_{l,FP,NS} + \alpha_{l,FP,S}) \quad (\text{원문 Eq. 34})$$

(합 안에서 해 진폭이 $-\alpha_{l,FP,S}$ 로 들어가는 것은 오라클 통과 후이기 때문입니다.) 저자들은 실제 구현이 이 스케일링을 어떻게 하는지도 명시합니다: “ 2^n 개 진폭의 누산합을 n 비트 정수부를 갖는 고정소수점 수에 저장한 뒤, $(n - 1)$ 비트 우측 시프트를 적용” 합니다. 이때 2^{-f} 미만의 소수부 비트가 버려지므로

$$\epsilon'_{l,scaled} = \left(2^{-n+1} \sum_{h=0}^{2^n-1} \psi'_{l,FP}[h] \right) \bmod 2^{-f} \quad (\text{원문 Eq. 35})$$

가 되고, 스케일링 결과는 고정소수점 값과 그 오차의 합으로 쪼개집니다:

$$2^{-n+1} \sum_{h=0}^{2^n-1} \psi'_{l,FP}[h] = \alpha'_{l,FP,scaled} + \epsilon'_{l,scaled}. \quad (\text{원문 Eq. 36})$$

이걸 Eq. (33)에 대입하면

$$\psi''_{l,FP}[m] = \begin{cases} \alpha'_{l,FP,scaled} + \alpha_{l,FP,S} + \epsilon'_{l,scaled}, & m \in S, \\ \alpha'_{l,FP,scaled} - \alpha_{l,FP,NS} + \epsilon'_{l,scaled}, & \text{otherwise.} \end{cases} \quad (\text{원문 Eq. 37})$$

오차 벡터의 갱신. 오차 벡터에도 똑같이 확산을 적용합니다(원문 Eq. 38):

$$2^{-n+1} \sum_{h=0}^{2^n-1} e'_l[h] = 2^{-n+1} \left((2^n - n_s) \epsilon_{l,NS} - n_s \epsilon_{l,S} \right) = 2\epsilon_{l,NS} - 2^{-n+1} n_s (\epsilon_{l,NS} + \epsilon_{l,S}).$$

최종 점화식. 고정소수점 레지스터에 실제로 남는 값에서는 $\epsilon'_{l,scaled}$ 가 잘려 나가고, 잘려 나간 그만큼이 오차 쪽으로 넘어갑니다. 이 “장부 정리”가 다음 두 식입니다:

$$\psi_{l+1,FP}[m] = \psi''_{l,FP}[m] - \epsilon'_{l,scaled} = \begin{cases} \alpha'_{l,FP,scaled} + \alpha_{l,FP,S}, & m \in S, \\ \alpha'_{l,FP,scaled} - \alpha_{l,FP,NS}, & \text{otherwise,} \end{cases} \quad (\text{원문 Eq. 39})$$

$$e_{l+1}[m] = e''_l[m] + \epsilon'_{l,scaled} = \begin{cases} 2\epsilon_{l,NS} + \epsilon_{l,S} + \epsilon'_{l,scaled} - 2^{-n+1} n_s (\epsilon_{l,NS} + \epsilon_{l,S}), & m \in S, \\ \epsilon_{l,NS} + \epsilon'_{l,scaled} - 2^{-n+1} n_s (\epsilon_{l,NS} + \epsilon_{l,S}), & \text{otherwise,} \end{cases} \quad (\text{원문 Eq. 40})$$

초기값은 — 초기 진폭 $1/\sqrt{2^n}$ 자체가 이미 고정소수점으로 표현되지 않을 수 있다는 사실을 반영해서:

$$\psi_{0,FP}[m] = \frac{1}{\sqrt{2^n}} - e_0[m], \quad e_0[m] = \left(\frac{1}{\sqrt{2^n}} \right) \bmod 2^{-f}. \quad (\text{원문 Eq. 41})$$

Eq. (39)와 (40)을 $l = k$ 까지 반복하면 최종 상태 $|\psi_{k,FP}\rangle$ 와 누적 오차 $|e_k\rangle$ 가 나옵니다.

점근적 스케일링 — 여기가 결론입니다. n 이 충분히 크고 n_s 가 작으면 $2^{-n+1} n_s \approx 0$ 이므로 Eq. (40)이 크게 단순해집니다:

$$e_{l+1}[m] \approx \begin{cases} 2\epsilon_{l,NS} + \epsilon_{l,S} + \epsilon'_{l,scaled}, & m \in S, \\ \epsilon_{l,NS} + \epsilon'_{l,scaled}, & \text{otherwise.} \end{cases}$$

비해 오차는 매 반복마다 스케일링 오차 하나씩만 더해지는 단순 누적입니다:

$$\epsilon_{k,NS} = \epsilon_{k-1,NS} + \epsilon'_{k-1,scaled} = \dots = \epsilon_{0,NS} + \sum_{h=0}^{k-1} \epsilon'_{h,scaled}. \quad (\text{원문 Eq. 42})$$

Eq. (41)과 (35)로부터 $\epsilon_{0,NS} < 2^{-f}$ 이고 각 $\epsilon'_{h,scaled} < 2^{-f}$ 이므로, 항이 k 개 쌓이면

$$\epsilon_{k,NS} = O(2^{-f}) + O(k \cdot 2^{-f}) = O(2^{\frac{n}{2}-f}), \quad (\text{원문 Eq. 43})$$

여기서 Eq. (9)에 의해 $k \approx O(2^{n/2})$ 를 썼습니다. 즉 반복 횟수가 $\sqrt{2^n}$ 이라는 그로버의 자랑거리가, 오차 관점에서는 그대로 $\sqrt{2^n}$ 배의 오차 증폭으로 돌아옵니다.

해 오차는 더 사납게 자랍니다. Eq. (40)에서 $m \in S$ 일 때 $2\epsilon_{l,NS}$ 항이 붙는 게 결정적입니다 — 이미 $O(k \cdot 2^{-f})$ 로 자란 비해 오차가 매 반복마다 해 오차에 **또** 더해집니다. 이중 누적이죠:

$$\epsilon_{k,S} = \epsilon_{k-1,S} + (2\epsilon_{k-1,NS} + \epsilon'_{k-1,scaled}) = \dots = \epsilon_{0,S} + \sum_{m=0}^{k-1} (2\epsilon_{m,NS} + \epsilon'_{m,scaled}) = O(k^2 2^{-f}) = O(2^{n-f}) \quad (\text{원문 Eq. 44})$$

k 개의 항을 더하는데 각 항이 이미 $O(k \cdot 2^{-f})$ 이니 $k^2 = 2^n$ 배가 되는 것입니다. 정리하면:

$$\boxed{\epsilon_{k,NS} = O(2^{\frac{n}{2}-f}), \quad \epsilon_{k,S} = O(2^{n-f})}. \quad (\text{원문 Eq. 45})$$

직관적으로: **진폭이 커지도록 증폭되는 바로 그 메커니즘이 오차도 같이 증폭합니다.** 그로버가 해 진폭을 키우는 방식이 “비해 진폭들을 굵어모아 해 쪽으로 옮기는 것”이니, 비해 쪽에 붙어 있던 오차도 함께 해 쪽으로 실려 오는 셈입니다.

3.3 측정 확률분포에 미치는 영향 ★

원문 p.16-18

진폭 오차를 알았으니 이제 **측정 확률**의 오차로 번역합니다. 실제로 우리가 관측하는 것은 진폭이 아니라 확률이니까요. 척도로는 이상적 확률분포와 에뮬레이션된 확률분포 사이의 ℓ_2 거리를 씁니다:

$$\ell_2 = \sqrt{\sum_{m=0}^{2^n-1} (p[m] - p_{FP}[m])^2}. \quad (\text{원문 Eq. 46})$$

이상적 확률은 Eq. (4)에 Eq. (29)를 넣어 전개합니다:

$$p[m] = |\psi_k[m]|^2 = (\psi_{k,FP}[m] + e_k[m])^2 = (\psi_{k,FP}[m])^2 + 2\psi_{k,FP}[m]e_k[m] + (e_k[m])^2. \quad (\text{원문 Eq. 47})$$

그런데 **제공 자체도 고정소수점 정밀도를 넘어갑니다** (f 비트끼리 곱하면 $2f$ 비트가 필요하니까요 — 2.3절 Fig. 4의 그 이야기입니다). 이 불일치를 **제공 반올림 오차** $e_{sq}[m]$ 로 잡아 줍니다:

$$p_{FP}[m] = (\psi_{k,FP}[m])^2 - e_{sq}[m], \quad e_{sq}[m] = \left((\psi_{k,FP}[m])^2 \right) \bmod 2^{-f}. \quad (\text{원문 Eq. 48, 49})$$

따라서 **확률의 총 오차는 세 항으로 이루어집니다**(원문 Eq. 50, 51):

$$e_p[m] = \underbrace{e_{sq}[m]}_{\text{제공 시 버림}} + \underbrace{2\psi_{k,FP}[m]e_k[m]}_{\text{교차항}} + \underbrace{(e_k[m])^2}_{\text{2차항}}.$$

각 항의 크기 산정. Eq. (49)에서 $e_{sq}[m] = O(2^{-f})$ 는 바로 나옵니다. 비해 진폭의 크기는 Eq. (19)의 부등식을 써서 다음과 같이 눌러집니다:

$$|\alpha_{k,FP,NS}| \approx |\alpha_{k,NS}| = \frac{|\cos((2k+1)\theta)|}{\sqrt{2^n - n_s}} < \frac{\sin \theta}{\sqrt{2^n - n_s}} = \frac{\sqrt{n_s}}{\sqrt{2^n} \sqrt{2^n - n_s}}, \quad \theta = \sin^{-1} \sqrt{\frac{n_s}{2^n}}. \quad (\text{원문 Eq. 52})$$

즉 $|\alpha_{k,FP,NS}| = O(2^{-n})$ 입니다. 반대로 해 진폭은 $|\alpha_{k,FP,S}| \approx n_s^{-1}$ 로, n 과 f 에 대해 **사실상 상수** $O(1)$ 입니다. (그로버가 성공적이라면 해 진폭이 1 근처여야 하니 당연한 결과입니다.) 이제 재료가 다 모였습니다:

$$\begin{aligned} \epsilon_{sq,NS} &= O(2^{-f}), & \epsilon_{sq,S} &= O(2^{-f}), \\ |\alpha_{k,FP,NS}| &= O(2^{-n}), & |\alpha_{k,FP,S}| &= O(1), \\ \epsilon_{k,NS} &= O(2^{\frac{n}{2}-f}), & \epsilon_{k,S} &= O(2^{n-f}). \end{aligned}$$

비해 상태의 확률 오차:

$$\epsilon_{p,NS} = O(2^{-f}) \pm O(2^{-n}) O(2^{\frac{n}{2}-f}) + O(2^{2(\frac{n}{2}-f)}) = O(2^{-f}) \pm O(2^{-\frac{n}{2}-f}) + O(2^{n-2f}). \quad (\text{원문 Eq. 53})$$

$n < f$ 이면 $2^{n-2f} < 2^{-f}$ 이므로 첫 항이 지배해서

$$\epsilon_{p,NS} = O(2^{-f}). \quad (\text{원문 Eq. 54})$$

(이 $n < f$ 조건은 실무에서 거의 항상 만족합니다 — 4.2절 공식이 $f_{\min} \approx n + 9$ 이상을 요구하니까요.) 재미있는 점은 비해 확률 오차를 지배하는 게 **누적된 진폭 오차가 아니라 그냥 마지막 제곱에서 잘리는 2^{-f}** 라는 것입니다. 비해 진폭 자체가 $O(2^{-n})$ 으로 워낙 작아서 교차항이 늘리기 때문입니다.

해 상태의 확률 오차:

$$\epsilon_{p,S} = O(2^{-f}) + O(1) O(2^{n-f}) + O(2^{2(n-f)}) = O(2^{n-f}). \quad (\text{원문 Eq. 55})$$

여기서는 **교차항 $2\alpha_{k,FP,S} \epsilon_{k,S}$ 이 지배합니다.** 해 진폭이 $O(1)$ 로 크니 누적 진폭 오차가 그대로 확률 오차로 넘어옵니다.

최종 ℓ_2 . $e_p[m]$ 도 Eq. (23)의 두 값 구조를 따르므로 Eq. (46)의 합이 두 덩어리로 정리됩니다:

$$\ell_2 = \sqrt{\sum_{m=0}^{2^n-1} (e_p[m])^2} = \sqrt{(2^n - n_s) \epsilon_{p,NS}^2 + n_s \epsilon_{p,S}^2}. \quad (\text{원문 Eq. 56})$$

$n_s \ll 2^n$ 을 가정하고 Eq. (54)~(55)를 넣으면

$$\ell_2 = \sqrt{O(2^n) O(2^{-2f}) + O(2^{2(n-f)})} \approx \sqrt{O(2^{2(n-f)})} = O(2^{n-f}). \quad (\text{원문 Eq. 57})$$

이 마지막 정리에 주목할 가치가 있습니다. 두 항이 경쟁합니다. 비해 항은 상태가 2^n 개나 되지만 각각의 오차가 $O(2^{-f})$ 로 작아서 합치면 $\sqrt{2^n \cdot 2^{-2f}} = O(2^{n/2-f})$ 입니다. 해 항은 상태가 n_s 개뿐이지만 각각의 오차가 $O(2^{n-f})$ 로 커서 $O(2^{n-f})$ 입니다. $2^{n-f} \gg 2^{n/2-f}$ 이므로 **소수의 해 상태가 다수의 비해 상태를 압도합니다.** 결론적으로:

f 를 1 늘리면 ℓ_2 는 절반이 되고, n 을 1 늘리면 ℓ_2 는 2배가 됩니다 (n_s 가 2^n 에 비해 작다는 전제 하에). 정확도를 유지한 채 큐비트를 1개 늘리려면 소수부 비트도 정확히 1개 늘려야 한다는 뜻입니다.

4. 실험 검증과 정밀도 설계 지침

원문 p.18-24

4.1 오차 공식의 검증

원문 p.18-23

무엇을 검증하는가. 앞에서 강조한 논점이 여기서 다시 나옵니다. 목표는 그로버의 이상적 최종 상태(Eq. 18)를 재계산하는 게 아니라, **이상적 게이트 하에서 반복 갱신을 수행하는 고정소수점 에뮬레이터의 반복 단위 산술 거동을 검증하는 것**입니다. 반올림은 반복되는 확산 스케일링 단계마다 생기므로, 닫힌 형태 식을 한 번 계산해서는 절대 드러나지 않습니다 — 루프를 실제로 돌려야 합니다.

Algorithm 1 (원문 p.19) — 측정된 ℓ_2 를 구하는 절차. 세 개의 프로시저로 되어 있습니다.

GROVER_FIXED(n, f, ns) — 고정소수점 경로:

```
int amp_NS, amp_S = floor(2^(-n/2 + f)) // 초기 균등중첩, 이미 여기서 버림 발생
int k = round((pi/4) * sqrt(2^n / ns))
for i = 0 to k-1:
    amp_S = -amp_S // 오라클: 해 진폭 부호 반전
    sum = (2^n - ns) * amp_NS + ns * amp_S // 전체 진폭 누산
    amp_NS = (sum >> (n-1)) - amp_NS // 확산: 2^{-n+1} * sum - amp ← 여기서 버림
    amp_S = (sum >> (n-1)) - amp_S
return amp_NS, amp_S
```

GROVER_DOUBLE(n, ns) — 배정밀도 부동소수점 기준값 경로. 완전히 같은 흐름이되 sum >> (n-1) 대신 $2^{-n+1} * sum$ 을 쓰고 초기값도 $2^{-n/2}$ 를 그대로 씁니다. 같은 반복 실행 흐름을 유지하면서 고정소수점 버림만 제거했기에, 소수부 양자화의 효과만 깨끗하게 분리됩니다.

GROVER_L2(n, f, ns) — 두 결과를 비교:

```
prob_fixed_NS = (double)((amp_NS^2) >> f) * 2^(-f) // 제곱 후 다시 f 비트 버림 (Eq. 48-49)
prob_fixed_S = (double)((amp_S^2) >> f) * 2^(-f)
prob_NS = amp_NS^2 ; prob_S = amp_S^2 // 기준 확률
return sqrt((2^n - ns)*(prob_NS - prob_fixed_NS)^2 + ns*(prob_S - prob_fixed_S)^2) // Eq. 46/56
```

Algorithm 2 (원문 p.20) — 3.2~3.3절의 이론 공식을 그대로 코드로 옮긴 것입니다. (1) Eq. (39)와 (40)을 반복해 최종 고정소수점 진폭 ($\alpha_{k,FP,NS}, \alpha_{k,FP,S}$) 과 누적 오차 ($\epsilon_{k,NS}, \epsilon_{k,S}$) 를 구하고, (2) Eq. (51)로 확률 오차 ($\epsilon_{p,NS}, \epsilon_{p,S}$) 를 계산한 뒤, (3) Eq. (56)에 넣어 이론 ℓ_2 를 냅니다. 주목할 점은 이 알고리즘이 **big-O가 아니라 실제 수치를** 계산한다는 것입니다 — mod 연산으로 ϵ_{scaled} 를 매 반복 정확히 뽑아내고 $2^{-n+1}n_s$ 항도 근사 없이 그대로 유지합니다.

실험 조건. $n = 8, 12, 16, f = 16, 17, \dots, 40, n_s = 1$. 정수부 폭은 모든 실험에서 $i = n$ 으로 고정해 오버플로가 없도록 했습니다.

결과 1 — 이론과 측정이 완전히 일치. 저자들의 표현으로는 두 값이 “exactly identical across all tested parameter combinations”입니다. 그래서 원문 Fig. 6(p.21)에는 n 하나당 곡선이 **하나씩만** 보입니다(측정 곡선과 이론 곡선이 완전히 겹쳐서). 이 완전 일치는 사실 놀랄 일은 아닙니다 — Algorithm 2가 근사 없는 결정론적 재현이니까요. 다만 이는 3.2절의 점화식 유도에 대수적 실수가 없음을 확인해 줍니다.

결과 2 — 지수적 감소. 원문 Fig. 6(p.21)은 세로축이 로그(10^0 에서 10^{-10} 근방까지), 가로축이 f (16~40)인 반로그 그래프입니다. $n = 8$ (파랑, 원), $n = 12$ (주황, 사각), $n = 16$ (초록, 삼각) 세 곡선이 **모두 거의 완벽한 직선**이며 서로 평행하게 위아래로 배치됩니다. 반로그에서 직선 = 지수적 감소이니, Eq. (57)이 예측한 2^{-f} 감소가 그대로 확인됩니다. 곡선 사이의 일정한 수직 간격은 n 의 효과가 f 와 무관한 곱셈 상수임을 뜻합니다.

이를 수치로 못박기 위해 폭 $w_{prec} = 4$ 구간마다 평균 스케일링 비율을 정의합니다:

$$r_{prec}(a) = \left(\frac{\ell_2|_{f=a}}{\ell_2|_{f=a-w_{prec}}} \right)^{\frac{1}{w_{prec}}}, \quad r_{num_q}(f) = \left(\frac{\ell_2|_{n=16}}{\ell_2|_{n=8}} \right)^{\frac{1}{16-8}}. \quad (\text{원문 Eq. 58, 59})$$

$r_{prec} \approx 0.5$ 이면 “비트 하나당 오차 절반”, $r_{num_q} \approx 2$ 이면 “큐비트 하나당 오차 2배”라는 뜻입니다.

원문 Table 2 (p.22) — ℓ_2 오차와 스케일링 비율 (범위 폭 $w = 4$):

n		$f = 16$	$f = 20$	$f = 24$	$f = 28$	$f = 32$	$f = 36$
8	ℓ_2	1.62×10^{-3}	1.04×10^{-4}	4.46×10^{-6}	2.78×10^{-7}	1.96×10^{-8}	1.06×10^{-9}
	$r_{prec}(f)$	—	0.5034	0.4551	0.4997	0.5152	0.4820
12	ℓ_2	2.25×10^{-2}	1.68×10^{-3}	1.31×10^{-4}	7.49×10^{-6}	4.05×10^{-7}	2.21×10^{-8}
	$r_{prec}(f)$	—	0.5225	0.5287	0.4890	0.4821	0.4835
16	ℓ_2	4.94×10^{-1}	2.98×10^{-2}	1.68×10^{-3}	1.14×10^{-4}	7.60×10^{-6}	5.24×10^{-7}
	$r_{prec}(f)$	—	0.4957	0.4869	0.5112	0.5078	0.5125
	$r_{num_q}(f)$	2.0442	2.0285	2.0981	2.1220	2.1067	2.1724

굵게 표시한 $n = 12, f = 28$ 의 7.49×10^{-6} 이 4.2절에서 비례상수를 결정하는 데 쓰이는 **데이터셋 중간점**입니다.

이 표에서 읽어야 할 것들. 첫째, r_{prec} 가 모든 구간에서 0.455_{0.529}로 **0.5 근방에 붙어 있습니다** — 비트 하나당 정확히 절반. 원문 Fig. 7(p.23)은 범위로 그린 그래프인데, 세 곡선 모두 0.5 선 주변에서 거의 평평하게 붙어 있어 시각적으로도 확인됩니다. 둘째, r_{num_q} 는 2.03~2.24로 **2 근방**입니다 — 큐비트 하나당 2배. 다만 f 가 커질수록 2.04 → 2.24로 살짝 상승하는 경향이 보이는데, 이는 Eq. (57)의 근사에서 무시한 비해 항 $O(2^{n/2-f})$ 등이 f 가 클 때 상대적으로 고개를 드는 것으로 이해할 수 있습니다. 어쨌든 실용 범위에서는 “2배” 규칙이 충분히 잘 맞습니다. 셋째, $n = 16, f = 16$ 에서 $\ell_2 = 0.494$ 라는 숫자를 **눈여겨보세요**. 확률분포 오차가 0.494면 사실상 에뮬레이션이 무너진 것입니다. $n \geq f$ 인 영역에서는 고정소수점 에뮬레이터가 아무 의미 없는 답을 낸다는 경고입니다.

4.2 목표 정확도를 위한 최적 정밀도 선택 ★★

원문 p.23-24

이 절이 논문에서 실무자가 가져갈 전부입니다. Eq. (57)에서 비례상수 a 를 붙여 등식으로 씁니다:

$$\ell_2 = a \cdot 2^{n-f}. \quad (\text{원문 Eq. 60})$$

a 를 확정하기 위해 4.1절 실험 데이터셋의 중간점 $(n, f, \ell_2) = (12, 28, 7.492 \times 10^{-6})$ 을 대입하면 $a = 2^{-1.03}$ 이 나옵니다. 따라서

$$\ell_2 = 2^{n-f-1.03}, \quad \text{즉} \quad f = n - \log_2 \ell_2 - 1.03. \quad (\text{원문 Eq. 61, 62})$$

목표 최대 오차 $\ell_{2,\max}$ 가 주어지면 필요한 최소 정수 소수부 비트는:

$$f_{\min} = \lceil n - \log_2 \ell_{2,\max} - 1.03 \rceil \quad (\text{원문 Eq. 63})$$

이 한 줄이 논문 전체의 결론입니다. 읽는 법: 필요한 소수부 비트 = (큐비트 수) + (목표 정확도의 이진 자릿수) - 1.03, 올림. 실용적으로 외울 만한 형태로 바꾸면,

- n 이 1 늘 때마다 f 도 1 늘려야 합니다. (정확도 유지 조건)
- 정확도를 10배 높이려면 f 를 약 3.32비트 ($= \log_2 10$) 늘려야 합니다.
- $\ell_{2,\max} = 10^{-3}$ 이면 $f_{\min} \approx n + 9$, 10^{-5} 이면 $\approx n + 16$, 10^{-7} 이면 $\approx n + 22$ 입니다.

원문 Table 3 (p.24) — 공식이 낸 $f_{\min}$ 과, 그 값을 실제 FP-Grover 에뮬레이터에 넣었을 때 나온 ℓ_2 :

n	$\ell_{2,\max} = 10^{-3}$ 의 $f_{\min}$	실제 ℓ_2	$\ell_{2,\max} = 10^{-5}$ 의 $f_{\min}$	실제 ℓ_2	$\ell_{2,\max} = 10^{-7}$ 의 $f_{\min}$
8	17	8.271×10^{-4}	24	4.459×10^{-6}	31
12	21	8.742×10^{-4}	28	7.493×10^{-6}	35
16	25	8.778×10^{-4}	32	7.599×10^{-6}	39

모든 칸에서 실제 ℓ_2 가 목표 $\ell_{2,\max}$ **아래**에 있습니다. 그리고 낭비도 크지 않습니다 — 예컨대 10^{-3} 목표에 실제 8.8×10^{-4} 면 여유가 채 1비트도 안 됩니다. 표를 세로로 읽으면 n 이 $8 \rightarrow 12 \rightarrow 16$ 으로 4씩 늘 때 $f_{\min}$ 이 $17 \rightarrow 21 \rightarrow 25$ 로 정확히 4씩 늘는 것도 보입니다. Eq. (63)의 n 계수가 1이라는 것의 실증이지요.

설계 함의. 저자들이 짚는 활용처는 두 갈래입니다. 소프트웨어 구현에서는 필요한 소수부 정밀도를 수용하는 **최소 메모리 풋프린트의 자료형**(int16 대 int32 등)을 고르는 근거가 됩니다. 하드웨어 구현에서는 $f_{\min}$ 에 맞춰 레지스터 폭, ALU, 데이터패스 폭을 재단해 정확도 제약을 지키면서 자원 사용을 최소화할 수 있습니다. 핵심은 **사전 계산(precompute)** 이라는 점입니다 — 시뮬레이션을 여러 번 돌려 비트폭을 더듬어 찾는 대신, 공식 한 번으로 답이 나옵니다.

5. 결론과 한계

원문 p.24-25

논문이 스스로 요약하는 성과는 이렇습니다. 그로버 반복 전반에 걸친 **단순하지만 엄밀히 정당화된 상태 표현**(두 진폭 구조)을 도입해, 반올림 오차가 연속된 게이트 연산을 거치며 전파·누적되는 과정을 정확한 수식으로 특성화했습니다. 그 결과 측정 확률분포의 오차가 $O(2^{n-f})$ 로 지수적으로 스케일함을 보였고, 수치 해석과 실제 고정소수점 QC 에뮬레이터 실험 양쪽으로 검증했습니다. 그리고 임의의 오차 허용치에 대해 최소 소수부 정밀도를 골라 주는 실용적 닫힌 형태 공식을 제공했습니다.

한계는 저자들이 먼저 인정합니다. 이 연구는 **의도적으로 알고리즘 특화**이며 임의의 양자 회로로 일반화되지 않습니다. 닫힌 형태 분석이 가능했던 것은 오로지 그로버의 강한 구조적 규칙성 — **저차원 상태 진화와 동일 연산자의 반복 적용** — 덕분입니다. 다른 구조화된 양자 알고리즘으로 이 정밀도 분석 방법론을 확장하는 것이 향후 과제로 남습니다.

여기에 독자가 스스로 챙겨야 할 한계를 몇 개 덧붙이면 좋겠습니다. 첫째, **오라클과 확산이 이상적이라고 가정**했으므로 실제 양자 소자의 잡음이나 게이트 오류는 이 분석 밖입니다(이건 에뮬레이터 설계 목적에는 오히려 적절한 분리입니다). 둘째, 상수 $a = 2^{-1.03}$ 은 $n_s = 1, n \in \{8, 12, 16\}$ **데이터의 중간점 한 점**에서 맞춘 값입니다. 다행히 Table 3이 세 가지 n 과 세 가지 목표치 전부에서 성립함을 보여 주지만, $n_s > 1$ 이거나 n 이 훨씬 큰 영역에서는 상수를 다시 확인할 필요가 있습니다. 셋째, ℓ_2 는 **확률분포 전체의 거리**이지 성공확률(success probability) 자체가 아닙니다. 측정 방식에 따라 요구되는 기준이 달라질 수 있습니다 — 이 점은 아래에서 다시 다룹니다.

한 가지 더. 이 논문은 **정밀도 대 자원의 트레이드오프를 명시적인 LUT/FF/BRAM/DSP 수치 표로는 제시하지 않습니다**. 자원 이야기는 “ $f_{\min}$ 에 맞춰 레지스터 폭과 데이터패스를 재단하라”는 정성적 지침 수준에 머뭅니다. 정량적 자원표를 기대했다면 이 논문이 아니라 저자들의 선행 연구(참고문헌 [32] Choi & Lee, AIMS Math. 9(11), 2024 — standalone FPGA 그로버 에뮬레이터)를 봐야 합니다. 이 논문의 기여는 “**몇 비트여야 하는가**”에 대한 답이지, “그 비트폭이 LUT 몇 개인가”에 대한 답이 아닙니다.

6. 이 프로젝트에 어떻게 쓰이나 ★★★

6.1 먼저, 우리 IP에 넣을 비트폭 숫자

$f_{\min} = \lceil n - \log_2 \ell_{2,\max} - 1.03 \rceil$ 을 우리 설계 범위에 그대로 계산해 둔 표입니다. $n = \text{NBITS}$, $f = \text{FRAC}$ 이고, 총 워드 폭은 $DW = i + f$ 에 정수부 $i = n$ (논문의 보수적 선택)을 쓴 값입니다.

n (NBITS)	$N = 2^n$	$f_{\min} (\ell_{2,\max} = 10^{-3})$	$f_{\min} (10^{-5})$	$f_{\min} (10^{-7})$	$DW = n + f_{\min} (10^{-5} \text{ 기준})$
2	4	11	18	25	20
3	8	12	19	26	22
4	16	13	20	27	24
6	64	15	22	29	28
8	256	17	24	31	32
10	1024	19	26	33	36
12	4096	21	28	35	40
16	65536	25	32	39	48

우리 현재 코드에 대입해 보면 — **아직은 이 공식이 물리지 않는 구간**입니다. 논문의 Algorithm 1을 그대로 옮겨 우리 파라미터로 돌려 본 결과입니다(원문 Table 2의 11개 값을 소수점까지 재현하는 것으로 구현을 검증한 뒤 측정했습니다).

grover_ip.v 는 $N=4$, $\text{NBITS}=2$, $DW=18$, $\text{FRAC}=16$ (Q2.16)입니다. Eq. (61)에 넣으면 $\ell_2 = 2^{2-16-1.03} \approx 3.0 \times 10^{-5}$ 가 나오지만, **실제로 돌려 보면 $\ell_2 = 0$ — 오차가 아예 없습니다**. $n = 2$

에서는 초기 진폭 $1/\sqrt{4} = 0.5$ 가 정확히 표현되고, 이후 모든 중간값이 큰 2의 거듭제곱의 배수로 유지되어 $(n-1) = 1$ 비트 우측 시프트가 **버리는 비트가 항상 0**이기 때문입니다. 최종 제곱($0.5^2 = 0.25$)도 소수부 2비트면 충분해 Eq. (48)의 제곱 버림조차 없습니다. $f = 8 \sim 32$ 전 구간에서 $\ell_2 = 0$ 임을 확인했습니다. $n = 4$ 도 $f \geq 16$ 이면 같은 이유로 정확합니다($f \leq 14$ 부터는 제곱 버림이 나타납니다).

grover_min_ip.v 는 $N=8$, $NBITS=3$, $DW=20$, $FRAC=16$ (Q4.16)입니다. 이쪽은 n 이 홀수라 $1/\sqrt{8} = 0.35355 \dots$ 가 애초에 표현되지 않고, 실측 $\ell_2 = 6.1 \times 10^{-5}$ 로 Eq. (61)의 예측 5.98×10^{-5} 와 잘 맞습니다. 정수부는 $i = DW - FRAC = 4 \geq n = 3$ 이라 논문의 오버플로 조건도 만족합니다.

여기서 교훈을 정확히 가져가야 합니다. Eq. (61)의 상수 $2^{-1.03}$ 은 $n = 12$, $f = 28$ 한 점에서 맞춘 **점근적 근사**이고, 논문이 검증한 범위는 $n \in \{8, 12, 16\}$ 입니다. $n \leq 4$ 처럼 진폭이 성긴 2진 격자 위에 놓이는 영역에서는 공식이 실제보다 오차를 **과대평가**합니다($n = 2$: 예측 3×10^{-5} vs 실제 0). 즉 **지금 우리 IP는 정말로 문제가 없는 구간에 있고, 이 논문의 공식은 우리가 n 을 8 이상으로 키우는 순간부터 필수**가 됩니다. 반대로 말하면, 지금 $n = 2, 3$ 에서 에뮬레이터가 잘 돈다고 해서 $n = 16$ 에서도 잘 돌 거라 추론하면 안 됩니다 — Table 2의 $n = 16$, $f = 16$ 에서 $\ell_2 = 0.494$ 였던 것을 기억하세요.

그런데 확장할 때가 진짜 문제입니다. 워드 폭이 $DW = n + f_{\min} \approx 2n + \log_2(1/\ell_{2,\max}) - 1.03$ 으로 n 에 대해 **2배** 속도로 자랍니다(정수부에서 n , 소수부에서 또 n). 메모리는 $2^n \times DW$ 비트이니, 10^{-5} 정확도 기준으로 $n = 8$ 이면 $256 \times 32 \text{ b} = 1 \text{ KB}$ 로 분산 RAM에도 들어가지만, $n = 16$ 이면 $65536 \times 48 \text{ b} \approx 3.1 \text{ Mb} \approx 393 \text{ KB}$ 로 중급 FPGA의 BRAM을 통째로 먹습니다. 즉 **BRAM 용량이 곧 큐비트 수의 상한**이 되고, 그 상한을 결정하는 데 $f_{\min}$ 이 직접 관여합니다. 이 계산이 나중에 “우리 보드로 몇 큐비트까지 가능한가?”에 답하는 출발점입니다.

정수부에서는 아껴 볼 여지가 있습니다. 논문 스스로 $i = n$ 이 최소가 아니라고 밝히고 있고, 2.3절의 코시-슈바르츠 한계가 실제로 요구하는 건 $|\sum \psi[m]| \leq 2^{n/2}$ 를 담을 정수부 — 대략 $n/2 + 1$ 비트입니다. 우리 코드의 reg signed [DW+NBITS:0] sum 은 누산기에만 여유 비트를 주고 진폭 배열 amp 자체는 DW 폭으로 유지하는 방식인데, 이게 오히려 논문의 $i = n$ 보다 앞뚫습니다. 다만 이건 우리가 검증할 몫이지 논문이 보장해 준 게 아니니, 비트폭을 줄일 때는 누산합의 최댓값을 시뮬레이션으로 확인하는 편이 안전합니다.

6.2 코드에서 고칠 것 — 시프트에서 1비트를 흘리고 있습니다

논문 Eq. (34)와 Algorithm 1이 명시하는 확산 스케일링은 $(n-1)$ **비트 우측 시프트 한 번**입니다:

amp_NS = (sum >> (n-1)) - amp_NS // 논문: 시프트 1회, 버림 오차 $< 2^{-f}$

그런데 grover_ip.v (75-85행)와 grover_min_ip.v (97-106행)는 둘 다 이렇게 되어 있습니다:

```
mean      = sum >>> NBITS;    // 평균                    ← n비트 버림
two_mean  = mean <<< 1;       // 2*평균               ← 다시 왼쪽으로
...
amp[i] <= two_mean - amp[i];
```

수학적으로는 $2 \times (\text{sum}/2^n) = \text{sum}/2^{n-1}$ 로 같지만, **고정소수점에서는 같지 않습니다**. $\text{sum} \ggg \text{NBITS}$ 가 하위 n 비트를 버려 결과를 2^{-f} 단위로 양자화한 뒤 $\lll 1$ 로 2배 하면, two_mean 의 LSB는 **항상 0**이 됩니다. 즉 two_mean 은 2^{-f} 가 아니라 2^{-f+1} 의 배수이고, 이 단계의 버림 오차 상한이 2^{-f} 가 아니라 $2 \cdot 2^{-f}$ 입니다. **소수부 비트 하나를 그냥 흘리고 있는 셈**입니다.

이 진단은 수치로 확인됩니다. 두 방식을 직접 돌려 비교하면, **repo** 방식을 f 비트로 돌린 결과가 **paper** 방식을 $f - 1$ 비트로 돌린 결과와 소수점 둘째 자리까지 일치합니다($n = 8, 12, 16 / f = 20, 24, 28$ 전 조합에서 비율 $0.99 \sim 1.00$). 즉 “1비트 손실”은 비유가 아니라 **문자 그대로 증가**입니다. ℓ_2 관점의 실제 손해는 $n = 6 \sim 16, f = 16 \sim 32$ 의 187개 조합에서 평균 **1.94배**(범위 $1.0 \sim 3.4$, 95.2%의 경우에서 나빠짐)였습니다. Eq. (57)이 예측하는 “비트당 2배”와 부합하되 정확히 2.00이 아닌 이유는, 버림이 결정론적이라 값에 따라 요동치기 때문입니다 — 원문 Table 2의 r_{prec} 가 0.5가 아니라 0.455~0.529로 흔들리는 것과 같은 성질입니다.

고치는 방법은 한 줄입니다 — 시프트를 나눠 하지 말고 논문처럼 한 번에:

```
two_mean = sum >>> (NBITS-1); // = 2 * (sum / 2^NBITS), 버림은 한 번만
```

FRAC 을 1비트 늘리는 것과 **수치적으로 동일한 효과**를 자원 증가 없이 얻습니다. NBITS ≥ 1 이면 안전하고, 우리 두 IP 모두 NBITS 가 2와 3이라 문제없습니다. mean 레지스터가 다른 데 안 쓰이면 같이 정리하면 됩니다.

다만 우선순위는 솔직하게 매기는 게 좋겠습니다. 6.1절에서 봤듯 현재 $n = 2$ 는 어차피 $\ell_2 = 0$ 이라 이 수정의 체감 효과가 없고, $n = 3$ 에서 $1.5 \times 10^{-4} \rightarrow 6.1 \times 10^{-5}$ 로 줄어드는 정도입니다. **이건 지금 급한 버그가 아니라, n 을 키울 때 반드시 들고 가야 할 수정**입니다. 큐비트를 늘릴수록 f 1비트는 그대로 메모리 2^n 비트로 환산되니까요.

6.3 초기 진폭 — 우리 상수가 논문 Eq. (41)과 정확히 맞습니다

논문 Eq. (41)은 초기 진폭 $1/\sqrt{2^n}$ 초차 고정소수점으로 표현되지 않아 $e_0 = (1/\sqrt{2^n}) \bmod 2^{-f}$ 만큼의 오차를 안고 시작한다고 말합니다. 우리 코드의 INIT_AMP 를 확인해 보면:

- grover_ip.v: $N = 4$, INIT_AMP = 18'sd32768. $1/\sqrt{4} = 0.5$ 이고 $0.5 \times 2^{16} = 32768$ 정확히 $\rightarrow e_0 = 0$. 초기 오차가 아예 없습니다.
- grover_min_ip.v: $N = 8$, INIT_AMP = 20'sd23170. $1/\sqrt{8} = 0.353553 \dots$, $\times 2^{16} = 23170.475 \rightarrow$ 버림해서 23170. $e_0 = 0.475 \times 2^{-16} \approx 7.2 \times 10^{-6}$.

즉 n 이 짝수면 $1/\sqrt{2^n} = 2^{-n/2}$ 라 초기 오차가 0이고, n 이 홀수면 $\sqrt{2}$ 가 끼어들어 반드시 오차가 생깁니다. 논문의 $e_0 < 2^{-f}$ 상한과 일치하며, Eq. (42)에서 이 $e_{0,NS}$ 가 끝까지 살아남아 누적되는 항이라는 점도 확인됩니다. 다행히 지배항은 k 번 쌓이는 ϵ'_{scaled} 쪽이라 초기 오차는 상대적으로 사소합니다.

6.4 grover_tb.v 로 논문을 재현해 보기

논문의 Algorithm 1은 사실상 **우리 IP의 수학적 모델**입니다. 두 IP가 진폭 배열 전체를 실제로 돌리는 반면 Algorithm 1은 두 진폭 구조(3.1절)를 이용해 스칼라 2개만 돌린다는 차이뿐, 산술은 동일합니다. 그러니 테스트벤치에서 이렇게 검증할 수 있습니다.

1. grover_ip.v 를 돌려 나온 amp[] 배열이 해 진폭 하나, 비해 진폭 $N - 1$ 개, 딱 두 값만 갖는지 확인 $\rightarrow$ 3.1절 정리의 하드웨어 검증. 이게 깨지면 오라클이나 확산에 버그가 있는 것입니다. 값싸고 강력한 assertion입니다.
2. NBITS 를 8 이상으로 올린 뒤 FRAC 을 스왑(16, 20, 24, 28...)하며 배정밀도 기준값과의 ℓ_2 를 계산 $\rightarrow$ 원문 Fig. 6과 같은 반로그 직선이 나오는지, 기울기가 비트당 0.5배인지 확인. $n = 2, 3$ 에서는 이

실험이 성립하지 않습니다(6.1절 — ℓ_2 가 0이거나 격자 효과에 지배됨). 논문이 $n \geq 8$ 만 다룬 데는 이유가 있습니다.

3. 6.2절 수정 전후로 ℓ_2 를 비교 → 수정 전 f 비트 = 수정 후 $f - 1$ 비트가 되어야 합니다(ℓ_2 기준 평균 1.94배 차이). 이 등가 관계가 1비트 손실의 가장 깔끔한 증거입니다.

이때 논문의 배정밀도 기준(GROVER_DOUBLE)을 쓰는 이유를 기억해 두면 좋습니다. 이상적 닫힌 형태 Eq. (18)과 비교하면 **반올림 오차와 Eq. (9)의 각도 잔차 δ 가 섞여** 버립니다. 같은 반복 흐름을 배정밀도로 돌린 값과 비교해야 고정소수점 양자화 효과만 깨끗이 분리됩니다.

6.5 주의할 점 — ℓ_2 가 우리의 합격 기준인가?

논문의 ℓ_2 는 **확률분포 전체의 거리**입니다. 그런데 우리 grover_ip.v 의 측정(92-104행)은 확률을 계산하지 않고 $\arg \max_i |amp[i]|$ 로 인덱스를 고릅니다. 이 방식에서 정답이 나오려면 필요한 조건은 훨씬 약합니다 — **해 진폭과 비해 진폭의 간격이 누적 오차보다 크기만 하면** 됩니다:

$$|\alpha_{k,FP,S}| - |\alpha_{k,FP,NS}| > \epsilon_{k,S} + \epsilon_{k,NS}.$$

$|\alpha_{k,S}| = O(1)$, $|\alpha_{k,NS}| = O(2^{-n})$ (Eq. 52)이므로 간격은 거의 $O(1)$ 이고, Eq. (45)의 $\epsilon_{k,S} = O(2^{n-f})$ 가 이보다 작기만 하면 됩니다. 즉 $f > n$ 정도만 되어도 **argmax는 정답을 맞힙니다**. 반면 $\ell_2 < 10^{-5}$ 는 $f \approx n + 16$ 을 요구합니다. **argmax 판독만 할 거면 논문 공식은 과한 기준**입니다.

그러니 목적에 따라 골라 쓰세요. 인덱스만 맞히는 데모라면 $f \approx n + 4$ 정도로도 충분할 수 있습니다. 반대로 **진폭이나 확률 값 자체를 출력**하거나, 그로버를 더 큰 계산의 서브루틴으로 쓰거나(예: grover_min_ip.v 의 Dürr-Høyer 라운드처럼 결과를 다시 입력으로 되먹이는 구조), 성공확률을 정량적으로 보고해야 한다면 논문의 ℓ_2 기준과 Eq. (63)을 그대로 쓰는 게 맞습니다. 특히 grover_min_ip.v 는 라운드를 거듭하며 상태를 재초기화하므로 진폭의 수치적 신뢰도가 더 중요합니다.

마지막으로 Eq. (9)의 함정 하나. 논문은 $k = \lfloor \frac{\pi}{4} \sqrt{2^n/n_s} \rfloor$ 을 **반올림**으로 정의하는데, $N = 4$ 에 넣으면 $\frac{\pi}{4} \times 2 = 1.571 \rightarrow k = 2$ 가 나옵니다. 그런데 $N = 4$ 에서 $\theta = 30^\circ$ 이므로 $k = 2$ 면 $(2k + 1)\theta = 150^\circ$ 로 **과회전**해서 성공확률이 $\sin^2(150^\circ) = 0.25$ 로 주저앉습니다. 정확한 최적값은 $k = 1$ 이고(90° , **성공확률 정확히 1.0**), 내림을 썼다면 맞았을 값입니다. 우리 grover_ip.v 의 R_ITSERS = 1 이 옳고 공식이 틀린 셈입니다. 논문이 $n \geq 8$ 만 다루므로 드러나지 않았을 뿐입니다. **n 이 작을 때는 Eq. (9)를 그대로 믿지 말고 $(2k + 1)\theta$ 를 직접 계산해 $\pi/2$ 에 가장 가까운 k 를 고르세요.** ($N = 8$ 은 $k = 2$ 에서 성공확률 0.945로 공식과 일치하니 grover_min_ip.v 의 R_GROVER = 2 는 문제없습니다.)

이 함정과 6.1절의 $\ell_2 = 0$ 은 사실 같은 이야기의 양면입니다. $n = 2$ 에서 $\ell_2 = 0$ 이라는 건 “고정소수점 에뮬레이터가 배정밀도 기준과 **완벽히 같은** 답을 낸다”는 뜻이지 “그로버가 정답을 맞힌다”는 뜻이 아닙니다. $k = 2$ 로 돌리면 두 경로 모두 사이 좋게 $\alpha_S = \alpha_{NS} = 0.5$ 라는 실패한 상태를 내놓습니다. ℓ_2 는 **산술 충실도의 척도이지 알고리즘 성공의 척도가 아닙니다** — 논문이 반올림만 분리하고 나머지는 이상적이라 가정했다는 2.3절의 전제가 여기서 그대로 드러납니다.

6.6 한 문단 요약

이 논문이 우리에게 주는 것은 **FRAC 파라미터를 정하는 근거** 하나입니다: $f_{\min} = \lceil n - \log_2 \ell_{2,\max} - 1.03 \rceil$ — 큐비트 하나당 1비트, 정확도 10배당 3.32비트. 지금 우리 IP는 $n = 2, 3$ 이라 FRAC=16 으로 오차가 0이거나 10^{-5} 수준이고, 사실 이 공식이 유효한 구간에 들어와 있지도 않습니다. **이 논문의 가치는 지금이 아니라 확장할 때 나옵니다** — n 을 8, 12, 16으로 키우는 순간 워드 폭이 $2n$ 속도로 자라 BRAM이 병목이 되고, FRAC 을 감으로 정하면 $n = 16, f = 16$ 의 $\ell_2 = 0.494$ 같은 재앙을 만나게 됩니다. 확장 전에 챙길 것 두 가지: (1) Eq. (63)으로 FRAC 을 먼저 계산하고, (2) 6.2절의 시프트 한 줄을 고쳐 공짜 1비트를 회수하세요.